ASYNCHRONOUS STATE RE-ANCHORING HANDSHAKE

SRAM Mailbox · VETO Interlock · Anti-Windup Protocol

Linux MPU (Untrusted System 1.5)		Real-Time MCU (Trusted System 1)
1. Proposes Action Chunk A_t NPU generates $H = 16$ trajectory chunk A_t from observation O_t and predicted q_{expected}.	SPSC Queue →	**1. Normal 1 kHz Spline Execution** MCU consumes waypoints from lock-free SRAM queue, interpolates C^2 splines to PWM.
2. Open-Loop Inference Running MPU computes next chunk A_{t+1} assuming previous chunk is executing smoothly.	← VETO	**2. Safety Veto Intervention** CBF detects obstacle ($h(x) < 0$). MCU **vetoes proposal**, overrides PWM, holds at q_{veto}.
3. State Divergence Detected $\|\|q_{\text{actual}} - q_{\text{expected}}\|\| > \epsilon_{\text{diverge}}$. If MPU kept streaming, violent snap-back would occur!	← VETO_ACTIVE	**3. Sets Atomic VETO Bit** MCU sets `VETO_ACTIVE = 1`, writes true $q_{\text{actual}} = q_{\text{veto}}$ to SRAM, trips NMI interrupt.
4. Flush Queue & Re-Anchor MPU flushes unexecuted waypoints, clears KV-cache, re-seeds planner from q_{actual}.	Chunk from q_{actual} →	**4. Resume Bumpless Motion** MCU clears `VETO_ACTIVE`, verifies C^2 boundary from q_{actual}, and resumes.

The Anti-Windup Guarantee: Without the asynchronous state re-anchoring handshake, a neural planner continues predicting from hallucinated positions while the robot is stopped, causing catastrophic snap-back when released. The shared SRAM mailbox protocol guarantees bumpless recovery.